

DDTA - Working Metamodel - Decision

NON CANONICAL WORKING DRAFT - REVISION 5

Construction candidate after the ThreatForge ownership-separation review and the first cross-cutting representation-contract decision. Concrete Doc-as-Code syntax, lifecycle storage, Requirement semantics, cross-document propagation implementation and ThreatForge refactoring remain deferred.

1 Purpose and evidence boundary

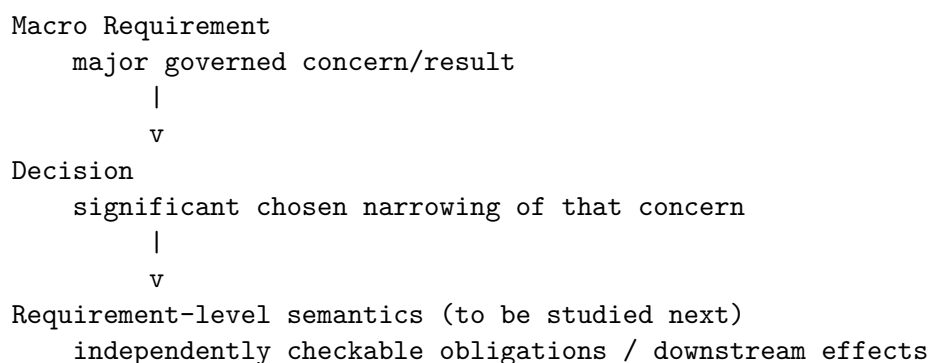
This study asks what a governed **Decision** means immediately below a Macro Requirement. Construction evidence remains intentionally limited to the sixteen historical ADRs of ThreatForge MR-0001 and MR-0002. MR-0003, MR-0004 and MR-0005 remain sequential holdouts and have not influenced this revision.

Revision 4 established that historical ThreatForge ADRs often conflate three semantic owners because the tool and the documentation model evolved together:

- DDTA method / metamodel;
- governed project;
- ThreatForge tool / ThreatForge project operations.

Revision 5 adds a separate cross-cutting conclusion: the metamodel should be able to drive a **complete canonical document shape** and deterministic executable projections without turning serialization details into document semantics.

2 Semantic position below Macro Requirement



The last level is intentionally not frozen here. **Requirement** remains the working name of the next document family, not a semantic assumption that may be silently embedded in ThreatForge ADRs.

3 Construction-candidate Decision semantics now CLOSED

The following items are **closed for the construction candidate**. They may be reopened only by an explicit counterexample from an independent example or a sequential holdout, with the reason recorded.

D-CLOSED-01 - Purpose of Decision

A Decision records one significant choice that resolves an unresolved problem or tension within one parent Macro Requirement and narrows that macro concern.

A Decision is not restricted to software architecture. Its subject may be a method, policy, convention, operating practice, semantic choice, representation choice, technology or architecture when that is the actual decision being governed.

D-CLOSED-02 - Minimal semantic core

The Decision-specific semantic core is:

```
Decision
|- title
|- context
|- decision
'- consequences
```

All four Decision-specific fields are semantically required and non-null. Parentage is an explicit semantic relation to the Macro Requirement, not prose duplicated inside each document. Life-cycle/status, author/date, paths and history remain common governance concerns outside this core.

D-CLOSED-03 - Decision-local Context

Context describes the unresolved local decision problem, tension or ambiguity that makes this specific choice necessary inside the already-governed parent concern.

If Context mostly explains why the parent MR exists, it is too high-level. Context may preserve historically true circumstances of the choice, but it must not become a cumulative copy of all inherited knowledge.

D-CLOSED-04 - One coherent chosen position

Decision states one coherent chosen position that can meaningfully be accepted, rejected or superseded as a conceptual unit.

Split diagnostic: if A can change while B remains valid and both have independently meaningful trade-offs, the historical ADR may contain more than one Decision.

D-CLOSED-05 - Consequence awareness

Consequences record relevant effects and trade-offs accepted because of the choice.

Benefit, Cost, Risk and Constraint remain useful authoring labels but are not frozen semantic subtypes.

D-CLOSED-06 - No semantic patching

If a competent reader can reasonably infer something false from Context + Decision, those sections must be rewritten. The model does not repair misleading semantics by accumulating exclusion

fields. Therefore Non-goals is not part of the Decision semantic core. It may remain an exceptional editorial technique outside the model when a document truly needs local clarification.

D-CLOSED-07 - No mandatory embedded Alternatives or Rationale fields

The construction corpus does not justify mandatory `alternatives[]` or a separate `rationale` field. Substantial alternatives may exist as separately governed candidate Decisions and lifecycle will later express acceptance/rejection/supersession. Reasoning is expected to be recoverable from:

Context + Decision + Consequences

This item can be reopened only if a counterexample demonstrates loss of necessary decision semantics.

D-CLOSED-08 - Lifecycle is not Decision-specific semantics

Acceptance, rejection, supersession, history and transition evidence belong to the later cross-cutting governance/lifecycle study. Historical Decisions may remain governed historical records without remaining current authoritative choices.

D-CLOSED-09 - Semantic-owner separation

A Decision must not silently transfer ownership of general model semantics to the tool that implements or supports them.

Working ownership model:

DDTA method / metamodel

owns general semantics and portable method choices

Governed project

owns concrete governed instances and project-specific choices

ThreatForge tool

supports authoring, resolution, validation, projection and workflow
without becoming semantic owner of the general model

ThreatForge project operations

govern development of ThreatForge itself
without becoming universalDDTA rules

Review test: **If ThreatForge were replaced by another tool, should the statement still be true?** If yes, it is probably a DDTA/model/method concern and cannot be justified merely as a ThreatForge product Decision.

4 Cross-cutting representation decisions now CLOSED

These decisions are **not Decision fields**. They constrain how governed document models are represented and projected once their semantics are defined.

R-CLOSED-01 - Complete-shape canonical representation

Every field declared by a governed document model is structurally present in every canonical representation of an instance of that model.

A missing declared field is a representation non-conformity. Optional semantic value does not imply optional structural presence.

For example, if a future model declares `assumptions` as nullable, a canonical instance carries an explicit value such as `assumptions: null`; it does not silently omit the field. For the current Decision model, `title`, `context`, `decision` and `consequences` are non-null.

R-CLOSED-02 - Explicit nullability and canonical empty values

Whether a field may have no semantic value is declared by the model, not inferred from omission.

The representation contract distinguishes at least:

- **missing field**: invalid when the model declares the field;
- **null**: the field is known to the model and its value is explicitly absent where nullability permits it;
- **empty collection**: a collection-valued concept is applicable and currently has zero members;
- **empty string**: not a generic substitute for semantic absence unless a future field type explicitly permits it.

Exact null/empty rules remain field-specific so that one generic sentinel does not erase semantic distinctions.

R-CLOSED-03 - Semantic / representation separation

Meaning, value domains, cardinalities, nullability and invariants belong to the canonical model semantics; material presence, order, headings, containers, mirrors and serialization mapping belong to governed representation profiles.

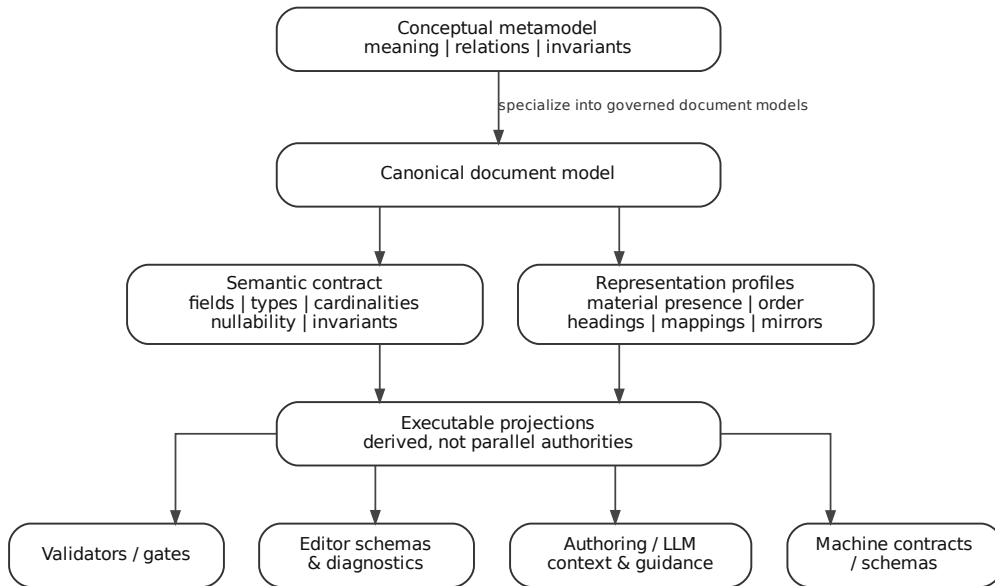
For example, Context preceding Decision preceding Consequences is useful and may be mandatory in the Markdown profile, but the ordering is a representation contract rather than an ontological claim about the Decision concept.

R-CLOSED-04 - Executable projections from one governed model

Validators, editor contracts, authoring assistance and machine-readable schemas should be deterministic projections of the governed canonical document model, not independently maintained parallel authorities.

This is a model-design property. It does not yet choose JSON Schema, a particular validator implementation, YAML syntax, Markdown grammar or ThreatForge architecture.

Model-to-executable-projection structure



The diagram deliberately separates **semantic contract** from **representation profiles**, then reunites them in derived executable projections. ThreatForge can consume those projections, but the projections remain derived from the governed model rather than becoming a second source of truth.

5 What is not fixed by the closed decisions

Semantic-owner separation does not decide whether DDTA will ultimately require Diataxis, a controlled vocabulary, implementation trace, a Security Requirement specialization or a particular Markdown reference grammar. Those remain subject-specific decisions in their proper future phase.

The representation decisions do not choose concrete serialization syntax. They only require that, when a canonical representation is defined, its field presence and nullability are unambiguous and its executable contracts derive from the same governed model.

This prevents the ThreatForge case study from becoming the accidental definition of the general metamodel and prevents validators/editors from becoming parallel semantic authorities.

6 Decision-to-downstream boundary - still partly OPEN

Working rule:

A Decision chooses; downstream governed documents make the chosen consequences operational and independently checkable.

The exact semantics of the next Requirement level are not yet closed, so revised ThreatForge ADRs avoid depending unnecessarily on classes such as Requirement, Functional Requirement or specific lifecycle states.

Redistribution diagnostic:

choice needed to explain what was adopted
-> remains in Decision

independently checkable obligation
-> candidate next-level document semantics

rollout / first increment / migration order
-> planning

separable choice with own trade-offs
-> another Decision candidate

Technical vocabulary is not forbidden. The issue is semantic ownership, not technicality.

7 Cross-document failure mode retained: persistent choices can be forgotten

The construction corpus exposed a problem that cannot be solved by adding topic-specific fields to Decision.

7.1 Vocabulary failure mode

Historical MR-0001/ADR-0002 chose a controlled vocabulary. The vocabulary source still exists, but the current MR-0001 Requirement registry has no direct descendants of ADR-0002, while later tooling governed under ADR-0004 consumes that vocabulary.

Conceptually:

Decision exists
shared governed knowledge exists
implementation partly exists

BUT

original downstream governance path is weak / no longer explicit

This is evidence that correctness cannot depend on programmers or LLMs remembering every old ADR.

7.2 Handoff positive counterexample

Historical ADR-0006 has dedicated downstream Functional/Governance Requirements, including continuity for a new LLM/session. Here the decision-to-realization path remained visible. The contrast is retained as empirical evidence for the future cross-document model.

8 Cross-document findings retained as OPEN research constraints

These are not Decision fields and not yet implementation mechanisms:

1. **No orphan normative decision** - a persistent accepted choice must not silently lose its downstream effect.
2. **Single-source governed knowledge** - shared vocabulary, taxonomies, patterns and similar knowledge should not be inherited by manual copy.
3. **Effective governed context** - a person/tool/LLM should be able to reconstruct the governed knowledge relevant to the current entity without rereading the repository by memory.
4. **Decision survivability** - persistent decisions should continue to influence work after the original contributor/context is gone.
5. **Recoverability** - knowledge being present somewhere is insufficient if its relevance cannot be discovered.
6. **Coverage awareness** - some cross-cutting obligations may require many consumers/artifacts rather than one implementation file.
7. **No inheritance-by-copy** - inherited/common context should be resolved through governed relations/projections rather than repeated in MR/Decision/next-level documents.

The exact relation types, algorithms and serialization remain OPEN until the combined MR -> Decision -> Requirement model exists.

9 Historical integrity and temporal semantics

Decision Context may preserve facts that were true when the choice was taken. Later evolution should normally create/revise/supersede Decisions through the future lifecycle model rather than rewriting old reasoning to look current.

Temporary rollout statements are different: when they describe only sequencing (first increment, later editor, migration order), they normally belong to planning rather than Decision semantics.

10 Review tests after revision 5

1. **Parent** - does the Decision narrow one parent MR?
2. **Decision-local Context** - does Context explain this unresolved choice rather than the whole MR?
3. **Explicit choice** - is the adopted position understandable without downstream implementation detail?
4. **Coherent choice** - can it reasonably be accepted/rejected/superseded as one unit?
5. **Consequence awareness** - are material effects/trade-offs visible?
6. **No semantic patching** - would a false reading require rewriting rather than a Non-goals patch?
7. **Semantic-owner test** - is the statement owned by DDTA/model, project, ThreatForge tool or ThreatForge project operations?
8. **Tool/model separation** - does ThreatForge support general semantics without silently defining them?
9. **Downstream boundary** - is independently checkable implementation detail descending rather than bloating the Decision?

10. **No ad-hoc cross-cutting fields** - are persistence problems kept as cross-document problems rather than topic-specific Decision fields?
11. **Complete-shape test** - can a consumer distinguish missing/non-conforming structure from an explicitly null value?
12. **Single-authority projection test** - can validators/editors/LLM guidance derive from the canonical model instead of copying semantic inventories?
13. **Historical integrity** - does the Decision preserve why the choice was taken without carrying temporary rollout status?
14. **Survivability challenge** - if persistent, can the overall model eventually explain how future authors/tools/LLMs discover its effect?

11 Open questions before Decision candidate closure

1. Does an independent facial-access example require separate Rationale despite D-CLOSED-07?
2. Does a non-software decision challenge the minimal core?
3. Where exactly is the threshold between precision intrinsic to the choice and next-level independently checkable detail?
4. Which historical large ADRs should ultimately split after ownership separation?
5. Are consequence labels semantic categories or only authoring aids?
6. What minimal cross-document relation/model preserves persistent decisions without copy inheritance?
7. How will the next Requirement model express pervasive/cross-cutting coverage?
8. How should effective governed context be projected for humans/tools/LLMs without becoming a new canonical source?
9. Which future document fields should be nullable, and which collection-valued fields should use a canonical empty collection rather than null?
10. Which representation-profile features are necessary to derive executable contracts without making renderer/editor concerns semantic?

12 Planned validation sequence

Before ThreatForge holdouts:

1. reread the revision-5 historical-vs-rewrite ADR comparison through the closed semantic and representation rules;
2. correct only failures exposed by that reading;
3. construct independent facial-access Decisions;
4. close the construction candidate if no unresolved semantic defect remains.

Then sequentially:

MR-0003 holdout

-> re-check construction set if corrected

MR-0004 holdout
 -> re-check previous cases if corrected
MR-0005 holdout

13 Explicitly deferred

- lifecycle/status value sets and transitions;
- registry/wrapper placement;
- concrete YAML/Markdown serialization syntax;
- Requirement-specific semantic model;
- effective-context propagation mechanism;
- concrete validator/editor implementation;
- migration/refactor of historical ThreatForge ADRs.

The current task remains semantic and model-oriented: determine what a Decision carries, define enough representation discipline to make the model executable by construction, and prevent the case-study tool from becoming an accidental owner of the metamodel it is helping to study.